

From Lean Proofs to Public Claims

Release checks and trust boundaries in one formal mathematics repository

Will Cook

July 2026

Abstract

Lean verifies that a proof establishes the formal statement written in the source. It does not verify that a README or a paper describes that statement faithfully. This paper explains how one public mathematics repository manages the gap. A maintainer-reviewed claim record states, for selected results, the public wording, the supporting Lean declarations, the finite domain, and the conclusion that remains open. A release checker then verifies recorded relationships: declaration names near their recorded source lines, the exact identity of the checked Lean source, required limitation wording, and freshly rebuilt generated files. Continuous integration runs that checker and the Lean build as two separate jobs. We follow one finite result through every layer, and we describe a deliberate exercise in which a false strengthening of the public wording passed every check because the relevant relationship had not yet been recorded. The checks preserve reviewed relationships; they do not interpret unrestricted prose, and the mathematical judgement remains human.

1 The publication gap

The repository at <https://github.com/wcook04/plectis-lean-erdos249-257> is a public Lean development around two unsolved problems in number theory, Erdős Problems 249 and 257. Both problems remain open. The project proves intermediate results, exact reformulations, and finite certificates around them; it does not claim a solution to either problem.

One theorem in the development illustrates the problem this paper is about. Lean has checked a finite certificate at each of 28 listed scales, the largest labelled $t = 64$: at each listed scale, a finite calculation, checked in full by Lean, establishes the required property at that scale. The registered open requirement asks for such certificates beyond every fixed cutoff. Whatever the largest checked scale is, one may choose a cutoff beyond it, and the finite list says nothing there. The finite list therefore does not settle the open problem.

A README can nevertheless drift. An edit can replace “28 checked cases, open beyond them” with wording that presents the finite cases as completing the open requirement. Every Lean proof remains valid under that edit, because the README is not part of any proof. No program in the repository decides whether a line of English has the same meaning as a formal statement.

The repository uses `docs/claims.json` as a *maintainer-reviewed claim record*. For each selected result the record states the public wording, its status, the Lean declarations that support it, the bounded domain it covers, and the registered conclusion that remains open. A release checker, `scripts/check_release.py`, then verifies that the recorded relationships still hold across the Lean source, the record itself, the authored public pages, and the generated files. The shortest accurate summary of the division of labour is:

Lean checks the formal proofs. A maintainer reviews what those proofs mean and how they may be described. The release machinery checks that the recorded relationships remain intact before a release.

Two boundaries should be fixed before any detail. First, the record covers *registered* claims, not every line of prose in the repository; software cannot preserve a relationship it was never pointed at, and Section 5 shows this limit operating in practice. Second, the repository does not technically force a second independent mathematician to approve a change. A single maintainer can edit a Lean statement, its claim record, and the public wording together, and every check can pass while the shared interpretation is wrong. The checks are drift detection for reviewed decisions; they are not a substitute for review, and they add no authority of their own.

2 The release workflow

Figure 1 shows the whole path. Solid boxes and lines are files and programs; dashed boxes and lines are human judgements. The figure has five visible parts: the Lean source; the maintainer-reviewed claim record; authored public documents; generated indexes and summaries derived from the record; and the release program and continuous-integration workflow that rerun the recorded comparisons.

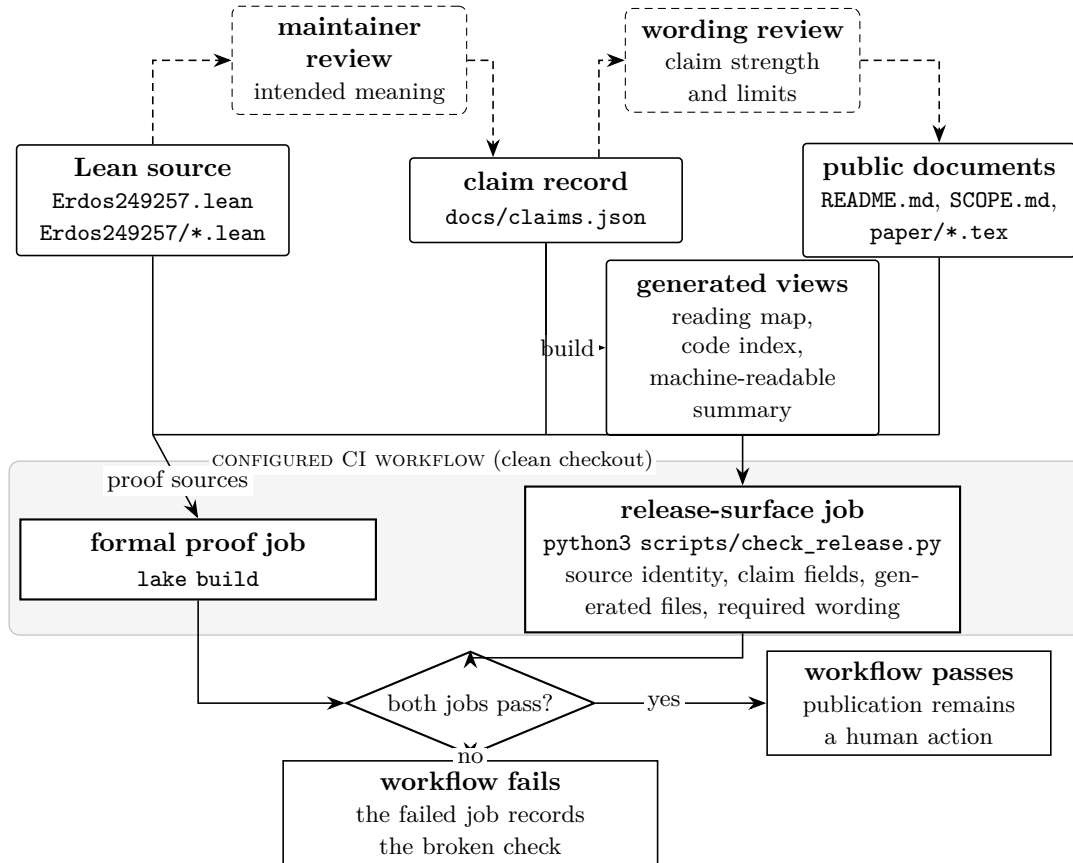


Figure 1: The configured checks for one revision of the repository. Dashed elements are human judgements; solid elements are files and programs. Maintainer review records the intended relation between the Lean source, the claim record, and the public wording. `lake build` checks the formal proofs. `scripts/check_release.py` compares only recorded relationships: source identity, claim fields, required wording, and freshly rebuilt generated files. Neither job reads unrestricted prose, and neither adds mathematical authority.

The two checks answer different questions. `lake build` asks whether Lean accepts the imported formal statements and proofs; it knows nothing about the README. `python3 scripts/check_release.py` asks whether the recorded relationships between the proofs, the

claim record, the public pages, and the generated files still hold; it confirms the identity of the Lean source but does not run Lean. After a mathematical change, project policy requires both. The workflow in `.github/workflows/lean.yml` runs them as two separate jobs on every push and pull request, in a clean checkout. A failed comparison makes `scripts/check_release.py` exit with an error, and the configured workflow treats that exit as a failing job. Blocking is therefore a property of the repository’s configuration, not of the script: the script can only refuse to succeed.

The generated views deserve one sentence of demystification. They select and reorganise recorded information (a reading map, a code index, a machine-readable summary) so that a reader or a program can enter the repository without opening every file. They create no new mathematics, each names the program that builds it, and a stale view is rebuilt by that owner rather than repaired by hand. The complete file map, ownership table, and maintenance commands live in the repository guide [ARCHITECTURE.md](#); this paper keeps only what the trust argument needs.

3 One claim from Lean theorem to public page

The architecture becomes concrete when one result is followed through every layer. We begin with its public meaning, not its internal name:

Lean has checked a finite certificate at each of 28 listed scales through $t = 64$. The registered open requirement asks for certificates beyond every fixed cutoff. The finite list does not settle it.

3.1 The formal evidence

The Lean module `Erdos249257/DiagonalPincerCertificatesT64.lean` defines the explicit list of scales,

$\{1, 2, 3, 4, 5, 7, 8, 9, 11, 13, 16, 17, 19, 23, 25, 27, 29, 31, 32, 37, 41, 43, 47, 49, 53, 59, 61, 64\}$,

and proves the assembling theorem `certifiedKill_diagonal_all_imported_through_t64`. Put $H_t = \text{lcm}(1, \dots, t)$. For every listed t , the theorem gives a finite checking depth d_t and a decidable arithmetic witness at (H_t, H_t, d_t) . The witness says that a finite weighted difference of totient values has a residue modulo a power of 2 outside a specified neighbourhood of zero. Imported modules prove the individual cases, and the terminal theorem combines them. The proofs use kernel-checked arithmetic computation, including `decide` and arithmetic normalisation. The root `Erdos249257.lean` imports the module, so `lake build` checks it.

The boundary is logical, not merely a matter of scale. The checked result says: for every t in the displayed finite set, the required witness exists. The open requirement says: for every cutoff T , some witness exists at a parameter beyond T . No finite list, however long or expensive to verify, implies the second statement without a further theorem that produces new witnesses. Thus $t = 64$ is the endpoint of this verified computation, not a frontier that Lean has proved will continue.

3.2 The reviewed record

Table 1 shows the substance of the corresponding entry in `docs/claims.json`. The entry also records an exact source line for each declaration; those coordinates are omitted from the table, and the full entry is in the file.

Field	Content
Claim identifier	<code>certified_kill_instances</code>
Status	<i>verified finite instance</i> : Lean checked stated finite inputs; this is not a score.
Public statement	“Kernel-checked certified kills for small periods and at 28 explicit lcm-diagonal scales through $t = 64$, plus finite certificate shards, each discharged by decide .”
Bounded domain	The listed small periods, the 28 scales above, and the parameters encoded by the finite certificate shards.
Supporting declarations	Five named Lean declarations with recorded modules and source lines, ending in <code>certifiedKill_diagonal_all_imported_through_t64</code> .
Remaining open	A link to the registered open requirement <i>unbounded certificate supply</i> : produce certified witnesses at unbounded parameters.

Table 1: The reviewed claim record entry for the worked example, with exact source coordinates omitted.

The claim identifier is a lookup handle for software, which is why it appears after the ordinary-language claim rather than in place of it. The status vocabulary is fixed in the record itself, and the release checker rejects a status outside that vocabulary. What no field can do is make the English faithful to the Lean: the maintainer judged that the public statement above describes the five declarations correctly, and the record stores the outcome of that judgement.

3.3 What the checker verifies here

For this claim, `scripts/check_release.py` verifies a specific, finite set of relationships. Each declaration name must appear in the Lean source within three lines of its recorded coordinate; this is a drift check on names and positions, not a check that the declarations entail the public wording. The checked Lean source itself must be exactly the formal-source checkpoint named in the claim record: the checker compares the supported root and the whole library directory against that recorded version and rejects the release if they differ or if untracked Lean files are present, so the verified names cannot refer to an older source than the one being shipped. The required limitation wording must remain present on the registered public pages. The generated views must equal a fresh rebuild from their sources. Presence, identity, and proximity are all the checker can test. Lean checks the theorem; the checker checks the recorded description around it; a person remains responsible for the claim that the two say the same thing.

4 What the checks establish

Table 2 states the guarantee and the non-guarantee of each layer. The separation exists because combining the layers in prose makes the guarantee sound stronger than it is.

Some of the release checker’s work is stronger than the phrase “consistency check” may suggest, and it is worth stating exactly. As a project-source policy, the checker rejects `sorry`, `admit`, new `axiom` declarations, and `native_decide`; the last exclusion keeps finite certificates on the repository’s kernel-checkable computation path. This lexical policy does not show that imported libraries use no classical axioms, and the pinned toolchain and imported definitions remain part of the trust base. The checker also compares the entire public proof surface byte for byte against the recorded formal-source checkpoint and rejects untracked Lean files. It rebuilds

Layer	What it can establish	What it cannot establish
Lean build and kernel	The written formal statements are proved from their imported definitions and assumptions, in the pinned toolchain.	That a formal statement is the one the author intended, or that any English description of it is faithful.
Maintainer review	A recorded judgement that a formal statement has the intended meaning and that selected public wording describes it within stated limits.	That Lean accepts the proofs; that every important claim was noticed and registered; its own independence or quality.
Release checker	That recorded names, coordinates, fields, links, required wording, source identity, and generated files agree with the declared rules, and that its own deliberately wrong examples still fail.	The meaning of unregistered prose; the completeness of the recorded checklist; the correctness of the human judgements it preserves.
Continuous integration	That both jobs ran in a clean checkout of the pushed revision and exited successfully.	Independent mathematical approval; anything beyond what the two jobs themselves establish.

Table 2: The trust boundary, one layer at a time.

the generated views and requires exact agreement rather than plausible freshness. It verifies the recorded source and PDF fingerprints of the shipped papers. And it carries its own negative examples: deliberately wrong inputs that must keep failing, so that a check cannot silently become vacuous. None of this reaches semantics. A fingerprint can prove two files identical; it cannot say which of two differing files expresses the right mathematics.

When a file changes, the path to a release is short to state. Edit the formal source and run `lake build`; review whether the statement, assumptions, or intended meaning changed, which is a mathematical judgement; if public meaning changed, update `docs/claims.json` under the review rules in `docs/methodology.json` and update the authored pages where readers meet the claim; rerun the named builders for the generated views; run `python3 scripts/check_release.py`; push, and the workflow repeats both jobs in a clean checkout. A prose-only edit skips nothing after the first step: if the words now make a stronger or different mathematical claim, the record and the review must change with them.

5 A boundary the checklist missed

The limits in Table 2 are not hypothetical. They were probed by a deliberate exercise, recorded in `docs/publication_evidence.json`, whose shape is worth describing exactly.

The retained maintainer record reports that, at a pinned checkpoint of the repository, ten deliberate false edits were applied to an isolated working copy, one at a time, with the copy restored between runs and the full release gate run after each edit. The edits targeted the seven recorded relationship families: status labels, record structure, source coordinates, paper link anchors, boundary wording, generated-view freshness, and size budgets. Representative edits upgraded a conditional status to “proved here”, deleted an open-boundary clause from the README, drifted a declaration coordinate by forty lines, introduced `native_decide` into a proof, and hand-edited a generated module count by one.

According to that record, nine of the ten edits were rejected. One escaped: an edit that strengthened a finite-evidence boundary clause in the README, replacing wording that the finite cases *do not supply* the open requirement with wording that they *complete* it. The formal proofs were untouched, and the release gate passed in full, because that clause was not among the recorded relationships. The error was outside the written checklist, and the checker preserved

exactly what it had been given.

The repair had two parts. The clause became a declared anchor: required wording on the public page, recorded in the checklist. And the checklist derives from each such anchor an opposing example: a copy of the page with the clause weakened or removed must be rejected. The intact text was then rechecked and passed; the same false edit now fails. The other nine edits were not rerun against the extended checklist, and no aggregate result is claimed for the repaired configuration.

This exercise demonstrates a coverage boundary, not a reliability score. Its ceilings should be stated as plainly as its outcome. The edits were authored by the author of the checker. Most relationship families were probed by exactly one edit. The exercise covers one repository, one maintainer, and one working style, with no comparison against ordinary continuous integration or careful manual review. The original run logs were not retained; the evidence file records the protocol, the outcomes, and these limitations, and a deterministic reconstruction of the edits can be replayed against the current checker, but the reconstruction is not the original evidence. What survives all of these ceilings is the design lesson:

The program can preserve a relationship only after a person has identified and recorded that relationship.

The residual risk follows the same line. Requiring a clause to be present does not detect a contradictory stronger claim elsewhere on the page. Coverage grows only by the route above: notice a relationship, record it, and give it an example that must fail.

6 Scope, reuse, and limits

Another formal project could reuse the pattern without copying any file format. Select the public claims whose strength matters. Record, for each, the ordinary-language meaning, the formal evidence, the bounded domain or assumptions, and the conclusion that remains open. Encode the relationships that should survive routine edits, giving every encoded relationship both an agreeing example and a deliberately disagreeing example. Configure the workflow so that a failed proof check or a failed comparison makes the configured workflow fail. The reusable content is the separation of authority and the explicit, reviewed bridge from formal result to public claim; the data shapes and scripts in this repository are local detail.

The limits are the mirror image of the design. The architecture does not establish a solution to either Erdős problem; that a formal statement is the statement the author intended; that software understood any unregistered prose; that the record is complete; that one maintainer’s review is independent or adequate; or that the design transfers to other projects with its behaviour intact. A large number of passing comparisons measures none of those things. What a green workflow run establishes is narrower: Lean accepted the formal proofs, and every configured comparison passed for the named revision. This paper is an architecture note with one bounded case study, not an empirical evaluation of the design.

7 Related systems

Proof blueprints connect plans and exposition to Lean declarations and track dependency and completion status: `leanblueprint` for authored blueprints [1], `LeanArchitect` for generating and maintaining them [2]. The claim record here sits later in the pipeline and answers a different question: not “what remains to formalise”, but “what may the finished formalisation be said to establish, and with what limits”. Lean Atlas supports judging whether a formal statement expresses the intended mathematics [3]; the present system takes that judgement as an input and preserves its recorded consequences. Isabelle/DOF places typed, checked structure inside

formal documents themselves [4]; this repository leaves most prose outside the proof assistant and checks an external record instead, which buys a clear human review surface at the cost of open-world coverage. Documentation-consistency systems such as CASCADE interpret prose to generate tests [5]; the design here takes the opposite point in that trade: it avoids automatic prose interpretation, while leaving unrecorded drift invisible. The deliberately wrong copies of Section 5 are known-answer regression fixtures in the tradition of mutation analysis [6, 7], not a mutation-score evaluation.

8 Conclusion

The workflow described here does not make prose formally true. It makes a selected set of proof-to-prose relationships explicit, reviewed, and mechanically persistent, and it makes the configured workflow fail when one of them breaks. The worked example shows both halves of that claim: the repaired checker accepts the intact wording and rejects the reconstructed false edit, while the earlier unrecorded boundary passed unnoticed. The architecture is therefore best understood as release discipline around formal proof, not as an extension of the proof assistant, and its value in any other project should be judged the same way: by whether it prevents consequential publication drift at an acceptable cost of maintaining the record.

A Reproducibility

The reviewed claim record is `docs/claims.json`. Its release block names the saved Git revision of the Lean source, and the release checker requires an exact match. The paper omits that changing identifier because the record is its single owner; copying it into prose would create a second value that can drift.

The paper inventory, with source and rendered fingerprints for each shipped manuscript and the commands that validate them, is `docs/publication_contract.json`. The record of the deliberate exercise in Section 5, including its protocol, outcomes, and limitations, is `docs/publication_evidence.json`; the deterministic reconstruction of its edits is described by `experiments/publication_mutations.json` and replayed with `python3 scripts/run_publication_mutations.py --verify-operators`. The original raw outputs of the historical runs were not retained, and the reconstruction does not claim to be them.

The papers build with Tectonic or a standard L^AT_EX installation via `make -C paper`. Buildability, not byte-identical output across TeX engines, is the claim. The shipped PDF is the tracked root copy, whose fingerprint the paper inventory records and checks.

References

- [1] P. Massot, *leanblueprint: a plasTeX plugin for formalization blueprints*, [project repository](#), accessed July 2026.
- [2] T. Zhu, P. Monticone, J. Avigad, and S. Welleck, *LeanArchitect: Automating Blueprint Generation for Humans and AI*, 2026, [arXiv:2601.22554](#).
- [3] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, [arXiv:2604.16347](#).
- [4] A. D. Brucker and B. Wolff, *Isabelle/DOF: Design and Implementation*, in *Software Engineering and Formal Methods*, 2019, pp. 275–292.
- [5] T. Kiecker et al., *CASCADE: Detecting Inconsistencies between Code and Documentation with Automatic Test Generation*, 2026, [arXiv:2604.19400](#).
- [6] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, *Hints on test data selection: Help for the practicing programmer*, *IEEE Computer* 11(4), 1978, pp. 34–41.
- [7] Y. Jia and M. Harman, *An analysis and survey of the development of mutation testing*, *IEEE Transactions on Software Engineering* 37(5), 2011, pp. 649–678.